第一篇的定位必须非常清楚：这不是一篇泛泛而谈的“AI优化模拟电路”论文，而是一篇围绕代理模型在优化后期失效、导致收敛不稳定与仿真浪费这一具体痛点展开的方法论文。整篇工作的目标不是简单提高一个回归模型的平均精度，而是构建一个不确定性感知的闭环优化系统，让模型能够知道自己什么时候可靠、什么时候不可靠，并据此决定是否调用真实仿真，从而在保证收敛质量的前提下显著减少仿真次数。只要这个目标始终不偏，整套实现就会有很强的顶会叙事一致性。

这篇工作的最小可行对象建议固定为一个经典且约束明确的模拟电路，例如两级运放或者LDO。两者都可以，但如果希望论文更容易讲清楚“后期失效”和“边界区域风险”这两个关键词，两级运放往往更适合，因为增益、GBW、相位裕度、功耗之间的trade-off更标准，phase margin临界区也更容易构造“失败案例”。因此，第一篇最稳妥的路线是选定单一拓扑，固定工艺与仿真环境，先把一个电路做深，而不是一开始追求多电路泛化。顶会并不要求一开始什么都做，只要求问题定义尖锐、方法闭环完整、实验足够有杀伤力。

整个工程实现必须拆成五个层级：仿真层、数据层、模型层、优化层、评测层。仿真层负责把设计变量送入SPICE环境并返回性能指标；数据层负责管理参数、性能、约束标签、失败标签和元信息；模型层负责训练代理模型并输出均值与不确定性；优化层负责根据模型结果和风险评分决定下一步采样与是否真实仿真；评测层负责做baseline对比、消融实验、失败案例分析和可视化。这个分层很重要，因为顶会论文里最怕系统耦合混乱，一旦代码结构不清晰，后期实验和论文图表都会非常痛苦。

GitHub仓库从一开始就要按论文结构来搭，而不是按“写到哪算哪”的方式来堆脚本。一个足够稳的目录结构可以写成下面这样：

```
analog-uaas/
├── README.md
├── LICENSE
├── .gitignore
├── environment.yml
├── requirements.txt
├── pyproject.toml
├── configs/
│   ├── circuit/
│   │   ├── opamp.yaml
│   │   └── ldo.yaml
│   ├── model/
│   │   ├── mlp_ensemble.yaml
│   │   ├── mlp_dropout.yaml
│   │   └── gnn_ensemble.yaml
│   ├── optimizer/
│   │   ├── random_search.yaml
│   │   ├── bo_baseline.yaml
│   │   └── uaas.yaml
│   └── experiment/
```

```
│       ├── exp_main.yaml
│       ├── exp_ablation.yaml
│       └── exp_ood.yaml
├── docs/
│   ├── proposal.md
│   ├── methodology.md
│   ├── simulation_pipeline.md
│   ├── data_schema.md
│   ├── experiment_protocol.md
│   └── paper_figures_plan.md
├── scripts/
│   ├── generate_initial_designs.py
│   ├── run_batch_simulations.py
│   ├── parse_raw_results.py
│   ├── build_dataset.py
│   ├── train_surrogate.py
│   ├── calibrate_uncertainty.py
│   ├── run_optimization.py
│   ├── evaluate_baselines.py
│   ├── plot_main_results.py
│   └── reproduce_paper.sh
├── src/
│   ├── __init__.py
│   ├── circuits/
│   │   ├── base_circuit.py
│   │   ├── opamp_design_space.py
│   │   ├── ldo_design_space.py
│   │   ├── netlist_writer.py
│   │   └── constraints.py
│   ├── simulation/
│   │   ├── simulator_interface.py
│   │   ├── spectre_runner.py
│   │   ├── ocean_runner.py
│   │   ├── job_manager.py
│   │   ├── result_parser.py
│   │   └── failure_checker.py
│   ├── data/
│   │   ├── schema.py
│   │   ├── dataset.py
│   │   ├── normalization.py
│   │   ├── split.py
│   │   └── cache.py
│   ├── models/
│   │   ├── base_model.py
```

```
│   │   ├── mlp.py
│   │   ├── gnn.py
│   │   ├── ensemble.py
│   │   ├── mc_dropout.py
│   │   └── calibration.py
│   ├── acquisition/
│   │   ├── base_acq.py
│   │   ├── ucb.py
│   │   ├── expected_improvement.py
│   │   ├── risk_aware_score.py
│   │   └── boundary_aware_score.py
│   ├── optimization/
│   │   ├── loop.py
│   │   ├── candidate_generator.py
│   │   ├── sampler.py
│   │   ├── active_learner.py
│   │   └── trust_controller.py
│   ├── evaluation/
│   │   ├── metrics.py
│   │   ├── benchmark.py
│   │   ├── ablation.py
│   │   ├── ood.py
│   │   └── visualization.py
│   └── utils/
│       ├── logger.py
│       ├── seed.py
│       ├── io.py
│       └── registry.py
├── netlists/
│   ├── templates/
│   │   ├── opamp_template.scs
│   │   └── ldo_template.scs
│   ├── generated/
│   └── models/
├── data/
│   ├── raw/
│   ├── processed/
│   ├── interim/
│   └── benchmarks/
├── results/
│   ├── logs/
│   ├── checkpoints/
│   ├── tables/
│   └── figures/
```

```
└── tests/
    ├── test_netlist_writer.py
    ├── test_parser.py
    ├── test_dataset.py
    ├── test_uncertainty.py
    └── test_optimization_loop.py
```

这个结构不是为了好看，而是为了保证四件事同时成立。第一，仿真可重复；第二，实验可追溯；第三，模型与方法替换方便；第四，论文图表能自动生成。顶会项目如果没有这种结构，很容易在实验阶段崩掉，因为每改一次模型都要连带重跑大量脚本，最后连哪张图对应哪组配置都说不清。

README必须从第一天就写，不要等最后补。README不只是仓库说明，而是整篇工作的“压缩版论文结构”。首页应该明确写出问题定义、方法概览、代码结构、快速开始、数据生成流程、优化流程、复现实验命令、主要实验表格的生成方式。docs目录中的文档也必须提前建好，其中`proposal.md`用于定义研究问题与创新点，`simulation_pipeline.md`说明网表模板、参数注入方式、仿真命令、结果解析与失败处理逻辑，`data_schema.md`明确每个样本记录哪些字段，`experiment_protocol.md`规定训练/验证/测试划分、对比方法、评价指标与随机种子，`paper_figures_plan.md`则从一开始就把打算画的图先列出来。这一点非常重要，因为很多学生最后写论文时才发现实验没按一个统一协议做，图表无法闭环。

仿真体系是整篇工作的地基，必须比模型更先搭好。一个完整的仿真体系至少包括设计空间定义、网表模板生成、批处理运行、结果解析、失败检测和任务调度六个模块。设计空间定义的作用是把“哪些参数可调、每个参数范围多大、哪些约束必须满足”明确写成程序对象，例如输入级晶体管W/L、负载管尺寸、尾电流、补偿电容等。网表模板生成则负责把这些参数填入Spectre或者SPICE模板文件中，生成一次仿真对应的具体网表。批处理运行模块要支持成批提交设计点并记录返回状态，因为实际仿真中一定会出现仿真收敛失败、结果字段缺失或者文件损坏。结果解析模块负责从输出日志或仿真结果中提取增益、GBW、phase margin、静态功耗、输出摆幅等指标，并统一写入结构化数据文件。失败检测模块必须将“仿真失败”和“性能不达标”区分开，因为这两者在方法上完全不同：前者是数值求解失败，后者是合法但无效的设计点。任务调度模块则负责并行仿真、缓存已有结果、避免重复运行。这一层做扎实之后，后面所有模型实验都会轻松很多。

数据体系不能只存“输入参数—输出性能”这么简单。为了支撑不确定性和主动学习，单条样本至少要包含以下信息：设计变量向量、标准化前原始参数、仿真环境配置、性能指标、是否满足spec、是否接近边界、是否仿真失败、仿真耗时、样本来源阶段（初始采样/主动学习补点/优化后期补点）、随机种子、时间戳。更进一步，建议为每个样本增加“约束违背向量”，即分别记录哪些指标没有满足。例如某个样本满足增益但不满足相位裕度，另一个样本满足相位裕度但功耗过高，这类标签在后续设计风险评分函数时非常有价值。顶会论文里如果能展示“模型误差主要集中在边界约束附近”，这种结论必须依赖样本级别的细标签，而不是只有总分。

模型体系上，第一篇最稳的主模型不是LLM，而是多输出回归模型。基础版建议先上MLP，因为参数型模拟电路在固定拓扑下，本质上就是一个数值向量到多目标性能向量的映射问题。为了做不确定性估计，最推荐的起点不是单个Bayesian Neural Network，而是Deep Ensemble。原因很简单：工程实现稳定，训练容易，审稿人接受度高，且在很多实际任务里比复杂贝叶斯网络更稳。可以先训练五个到十个初始化不同的MLP，共享同样的输入输出结构，预测时取均值作为性能估计

，取方差作为epistemic uncertainty近似。为了增强论文完整性，可以同时实现一个MC Dropout版本作为替代实现，用于展示方法并不依赖某一种具体不确定性机制。若后期精力足够，再补一个轻量GNN版本，用于说明结构信息是否带来额外收益，但这个不应该成为第一篇的主线，否则会分散叙事焦点。

第一篇的方法创新不能写成“使用ensemble估计不确定性”，因为这不新。真正的创新必须落在优化决策层。也就是说，单纯得到uncertainty还不够，关键是如何把uncertainty变成仿真调用策略。最合理的做法是定义一个风险感知采样分数，而不是直接套标准UCB或者EI。一个可行的思路是：对每个候选点，代理模型输出性能均值和不确定性；随后根据设计目标构造一个“可行性余量”或“边界距离”，例如离相位裕度下限还有多少、离功耗上限还有多少、离GBW目标还有多少；然后把不确定性与边界距离结合成一个风险指标。直观地说，如果某个点预测看起来很好，但它紧贴约束边界且不确定性很大，那么它就是高风险点，此时不应直接让模型主导迭代，而应优先调用真实仿真。如果某个点离边界较远且不确定性较小，则可让模型主导搜索而不必立即仿真。这样的方法故事才完整：第一篇不是“又一个不确定性估计模型”，而是“提出一种面向边界风险的自适应仿真调度策略”。

这里建议把核心方法命名为一个明确的框架，例如UAAS，即Uncertainty-Aware Adaptive Sampling。其内部流程可以写得很清楚：先在设计空间里生成初始样本，训练初始代理模型；在每轮优化中先由候选生成器提出一批候选点，再由代理模型预测其性能均值和方差；随后风险评估器计算每个点的风险感知分数；若分数低，则该点可以直接进入模型驱动的局部搜索；若分数高，则必须触发真实仿真；真实仿真结果返回后用于更新数据池，并定期重训或增量更新模型。这个框架的作用不是只在初期省仿真，而是尤其在后期避免被错误模型牵着走。这个“后期稳定性”的卖点一定要反复强调，因为这是让文章从普通代理优化论文变成更像顶会论文的关键。

主动学习模块要从“补点”升级成“围绕失效模式补点”。不能只是全局随机挑高不确定性点，那样很容易被审稿人认为只是标准active learning。更强的做法是让采样策略同时关注三类点：一类是高潜力点，即预测性能有望满足目标；一类是高不确定性点，即模型不确定；一类是边界敏感点，即靠近约束边界。真正有信息价值的点往往是后两类与第一类的交集，也就是“看起来值得相信，但又不能完全相信”的点。主动学习因此不再是泛化导向的全局补数据，而是服务于优化任务本身的定向采样。这一点写进proposal时会很加分，因为它强调了任务驱动，而不是盲目追求预测精度。

评价体系必须从第一天就围绕顶会标准来设计。只报告平均绝对误差是远远不够的，因为这只能说明模型是个还不错的回归器，不能说明方法能改善优化。第一组核心指标应该是优化效率指标，例如达到目标spec所需的总仿真次数、总墙钟时间、每次成功设计平均仿真预算。第二组是优化质量指标，例如最终最优点的综合性能、约束满足率、在多次随机初始化下成功收敛的比例。第三组是稳定性指标，例如后期迭代中目标函数是否震荡、是否频繁在边界附近往返、是否出现“模型说可行但仿真不可行”的误导事件。第四组则是不确定性质量指标，例如置信区间覆盖率、ECE式校准误差或者“高不确定性区域是否真的对应高误差”。只有这四组指标一起报告，论文的“可靠优化”主题才算真正立住。

Baseline的设计要非常克制而精准，不要贪多。最少需要四个对比对象。第一个是随机搜索或者拉丁超立方采样加真实仿真，这是最朴素基线。第二个是纯代理模型优化，即训练好模型后只用预测推进搜索，不做不确定性控制，这个用来证明后期失效问题确实存在。第三个是标准贝叶斯优化或常见采集函数方法，用来回应“这不就是BO吗”的质疑。第四个是“有不确定性但无边界风险

控制”的简化版本，用来做消融，证明方法不是简单因为用了ensemble而变强。再加上自家完整UAAS，这样就足够形成一个很稳的实验矩阵。顶会不怕baseline少，怕baseline没击中质疑点。

论文贡献必须写成“问题—方法—系统—实验”四段式，而不是一串技术名词。第一，系统性指出并验证了代理模型在模拟电路优化后期、尤其是接近设计约束边界时的失效模式，将其定义为可靠性瓶颈。第二，提出面向边界风险的不确定性感知自适应采样策略，使仿真调用从固定预算或静态规则变为由模型置信度和约束风险共同决定的动态行为。第三，构建了一个完整的模型—仿真协同闭环，将预测、不确定性评估、主动采样和在线更新统一到一个可复现实验框架内。第四，在典型模拟电路设计任务上证明该方法能够在保持或提高设计成功率的同时显著降低仿真次数，并在后期收敛阶段表现出更好的稳定性。这四条贡献如果都能被实验支撑，定会级别就具备了。

所谓“满足定会”，不是把标题写得像顶会，而是让整篇工作具备三个条件。第一，有明确且非平凡的failure mode，不能只是“我做了个新模型”。第二，方法必须直接针对该failure mode，而非技术堆砌。第三，实验必须能说明现有方法为什么会输，而不仅仅说明本方法平均值略高一点。只要把文章打造成“现有代理优化在边界区不可靠，UAAS通过风险感知仿真调度解决这一问题”的故事线，投稿到EDA顶会是有机会的。反过来，如果最后写成“使用ensemble和active learning提升精度”，那就很容易降格为常规机器学习应用文章。

具体实施步骤上，第一阶段只做仿真基础设施，目标是把一个拓扑的参数化网表、批量仿真、结果解析和失败检测全部跑通，并稳定地产生至少数千条设计样本。第二阶段完成数据清洗与特征规范化，同时明确设计目标与约束定义，建立训练/验证/测试协议和优化任务协议。第三阶段训练基础代理模型，先不做复杂花样，先把多目标回归和ensemble不确定性跑通，并验证误差在可接受范围内。第四阶段实现风险感知采样策略，把不确定性与边界距离结合起来，构成真正的方法主体。第五阶段搭建完整闭环优化系统，并跑所有baseline。第六阶段集中做三类实验：主结果实验、消融实验和失败案例实验。这里尤其要留出大量时间打磨“失败案例实验”，因为这类图最能说服审稿人，例如展示某个spec下纯代理方法在后期反复误判为可行，而UAAS能够及时触发仿真校正并稳定收敛。

从仓库文档角度，还必须额外写两类东西。一类是实验协议文档，明确每个实验用什么配置、多少随机种子、每个优化任务预算多少、何时重训模型、候选池如何生成。另一类是可视化说明文档，提前规划论文会需要哪些图。例如一张系统框架图，展示“代理预测—不确定性评估—风险判断—仿真调用—数据回流”的闭环；一张后期收敛曲线图，展示本方法比纯代理和BO更稳定；一张不确定性—误差相关性图，证明模型的不确定性确实有意义；一张失败案例图，展示某个边界区域中其他方法如何被误导。本质上，代码结构和论文结构应该从一开始就是同构的，这样后面写作时才不会散。

概括，不是要证明“模型可以彻底替代SPICE”，而是要证明“模型可以在知道自身局限的前提下有效管理SPICE调用”。这是一个非常重要的定位差异。前者很容易被工业界质疑，因为模拟电路天然对边界与极端条件敏感；后者则更现实，也更符合EDA工具演进方向。也正因为如此，这篇第一篇文章的真正价值，不只是节省一些仿真时间，而是给第二篇完整agent系统打下方法学基础。第一篇解决的是“模型什么时候值得信”，第二篇才能进一步解决“系统如何围绕这个判断来做更复杂的设计决策”。